Contents

SPECIFICATION — Synthetic Dataset Generator (Scenario Ground Truth, Controlled Language, `synthgen`+, uv)

- **Status:** v0.2 — SPEC_REVIEW findings F-001..F-010 integrated
- **Language:** Python 3.12 | `uv` | YAML/JSON specification | JSONL datasets | optional Ollama | CLI/GUI labels
- **Curriculum source:** `supplemental_docs/synthetic_dataset_generator_mvp.md` (§1 Purpose, §2 MVP Use Case, §3 Design Goals, §4 Architecture, §5 Separate Scenario Generation from Language Generation, §6 Scenario Generator, §7 Ground Truth Engine, §8 Natural-Language Generator, §9 Controlled Variation, §10 Constraint System, §11 Dataset

Schema, §12 Distribution Control, §13 Negative and Edge Cases, §14 LLM-as-Generator vs Template Generator, §15 Semantic Validation, §16 Deduplication, §17 Quality Metrics, §18 Reproducibility, §19 CLI Design, §20 Suggested Project Structure, §21 Development Sequence, §22 The Key Architectural Insight, §23 Bootcamp Learning Objective).

- **Scope of this document:** The authoritative contract for a deterministic-first synthetic dataset generator. It owns specification loading, scenario generation, constraints, deterministic ground truth, template/LLM realization, validation, deduplication, JSONL writing, quality reports, manifests, and CLI behavior. Optional GUI and Ollama generation are bounded adapters.
- **Normative language:** MUST/MUST NOT/SHALL/SHALL NOT = normative; SHOULD = strong; MAY = optional.
- **Principle: Generate parameters, calculate truth, then generate language.** A probabilistic language generator MUST NOT define or modify ground truth; validation MUST sit between generation and publication.

---

0. Intent and purpose

Synthetic data generation is an engineering pipeline, not an unconstrained request to "make 1,000 examples." This lab builds a generator from a declarative dataset specification so that semantic structure, ground truth, linguistic variation, validation, and provenance remain separately inspectable.

The central thesis is:

> **Synthetic data should be generated from a specification of the space you want to test, not from a request for examples.**

The system follows this deterministic/probabilistic boundary:

```
Dataset specification
        |
        v
Deterministic scenario space
        |
        +--> deterministic ground truth
        |
        +--> template language generation
        |
        +--> optional LLM language generation
                         |
                         v
                deterministic validation
                         |
                         v
                   JSONL dataset
```

The LLM MAY provide linguistic diversity, but it MUST receive a fixed scenario and MUST NOT modify the scenario or ground truth. Every accepted record MUST be traceable to a specification, seed, generator version, scenario, realization method, validation result, and deduplication decision.

Relationship to Chapter 31. The mortgage calculator's `evals/mortgage_questions.jsonl` is the motivating target dataset. Chapter 31 supplies the deterministic domain calculator and evaluation contract; this lab specifies a reusable generator that can produce similar datasets from a domain specification. The generator MUST depend on a calculator protocol rather than importing Chapter 31 implementation details.

Curriculum mapping. §1–§4 define purpose and architecture. §5–§7 define scenario/ground-truth separation. §8–§9 define language generation and variation. §10–§13 define constraints, schema, distributions, and

edge cases. §14–§16 define template/LLM trade-offs, semantic validation, and deduplication. §17–§18 define quality metrics and reproducibility. §19–§21 define CLI and implementation sequence. §22–§23 define the architecture lesson and learning objective.

Non-goals: unrestricted web-scale data collection, training a foundation model, autonomous domain discovery, unbounded LLM generation, hidden ground truth supplied by an LLM, semantic deduplication requiring a hosted embedding service, and a production annotation marketplace.

---

1. Actors and goals

Actor	Goals
Dataset author (`spec.py`)	Write a declarative YAML/JSON specification defining schema, categories, distributions, constraints, and generation methods.
Generator user (`cli.py`)	Generate, validate, inspect, preview, reproduce, and report on a dataset using explicit size, seed, adapter, and output options.
Scenario generator (`scenarios.py`)	Produce valid or deliberately invalid structured scenarios from bounded distributions and constraints.
Ground-truth engine (`truth.py`)	Calculate expected outcomes deterministically from each scenario through a registered domain calculator.
Language generator (`templates.py`, `llm.py`)	Render a scenario into user-facing language without changing scenario semantics. Templates are deterministic; the optional Ollama adapter is probabilistic.
Validator (`validators.py`)	Verify schema, intent, field preservation, expected outcome, scope, constraints, and scenario-to-language equivalence.
Deduplicator (`dedup.py`)	Reject exact duplicates and classify near duplicates according to the configured normalized representation.
Report writer (`writers.py`)	Emit JSONL records, quality reports, and reproducibility manifests with stable serialization.
Optional GUI user (`ui.py`)	Configure a generation run, preview records, inspect quality metrics, and select `Off`/`INFO`/`DEBUG` diagnostics without changing the generator core.
Ollama daemon *(optional external dependency)*	Generate paraphrases from supplied scenarios. It is never the source of ground truth.

---

2. Requirements (intent, high level)

ID	Statement
R-01	The system MUST load a declarative YAML or JSON dataset specification containing dataset metadata, output schema, categories, distributions, constraints, and generation configuration.

ID	Statement
R-02	The system MUST generate an abstract structured scenario before generating any natural-language realization.
R-03	The system MUST calculate ground truth deterministically from the scenario and MUST NOT use an LLM to calculate or modify ground truth.
R-04	The system MUST support valid, boundary, invalid, ambiguous, underspecified, unsupported, and adversarial scenario categories when declared by the specification.
R-05	The system MUST support bounded distributions for numeric fields, enumerated values, weighted categories, and relationship constraints.
R-06	The system MUST support deterministic template realizations with controlled variation in numerical, rate, term, and question phrasing.
R-07	The system MAY support Ollama-backed language realization, but the adapter MUST receive a fixed scenario and MUST NOT write ground-truth fields.
R-08	The system MUST validate every candidate record before publication and MUST reject or boundedly regenerate records that fail validation.
R-09	Semantic validation MUST compare the intended scenario with the parsed candidate realization, including intent, parameters, units, expected outcome, and scope.
R-10	The system MUST support exact deduplication by normalized text and MUST expose near-duplicate detection as a configured strategy with an explicit result.
R-11	The system MUST write accepted records as JSONL and MUST preserve user-facing input, ground truth, and generation metadata separately.
R-12	Every generated dataset MUST have a reproducibility manifest containing generator version, specification hash, seed, model/adapter configuration, generation parameters, and timestamp.
R-13	The system MUST produce quality metrics for category distribution, validation, rejection, duplicates, realization method, and generation cost when available.
R-14	The CLI MUST provide `generate`, `validate`, `stats`, `preview`, `reproduce`, and `inspect` subcommands.
R-15	The generator MUST support explicit dataset size, seed, output path, and category-distribution overrides without changing the specification file.
R-16	The mock/template path MUST run offline and deterministically; identical specification, seed, generator version, and options MUST produce byte-identical output.
R-17	The optional real-model path MUST be opt-in, identify the model and adapter in metadata, and convert dependency failures into structured errors.
R-18	The system MUST expose structured errors for invalid specifications, impossible constraints, generation exhaustion, validation failure, duplicate exhaustion, and model failures.
R-19	The system MUST provide a machine-readable report that explains every rejected record, including stage, reason, source scenario, and bounded candidate details.
R-20	The system MUST provide an optional PyQt5 GUI or a documented GUI boundary with the same generation operations and diagnostics levels as the CLI.
R-21	The CLI and GUI MUST provide opt-in diagnostics: omitted verbosity is quiet, bare `--verbose`/`INFO` emits metadata, and `DEBUG` additionally emits raw model prompts/responses; diagnostics MUST NOT alter generated records.
R-22	The system MUST allow a domain calculator to be substituted through a protocol without changing scenario, validation, reporting, or CLI layers.
R-23	The system MUST ensure that accepted records remain within the declared dataset scope and MUST reject an LLM realization that introduces unsupported concepts.
R-24	The system MUST preserve enough provenance to reproduce or audit an individual record without rerunning the LLM.

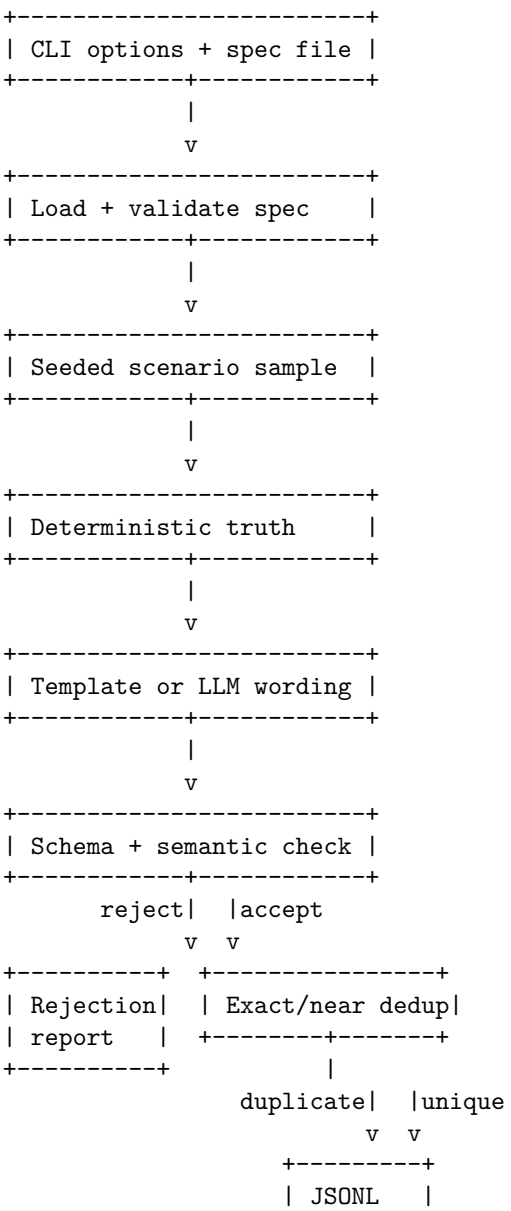
3. Behavior and state model

3.1 Lifecycle scope

A generation run is one bounded operation from specification load through manifest/report publication. It has five nested scopes:

1. **Specification scope:** parse and validate the declarative specification.
2. **Scenario scope:** sample one structured scenario using the run RNG.
3. **Record scope:** calculate truth, realize language, validate, deduplicate, and accept/reject one candidate.
4. **Dataset scope:** repeat bounded record attempts until the requested accepted size or attempt budget is reached.
5. **Artifact scope:** write JSONL, quality report, and manifest atomically or fail without claiming completion.

3.2 Generation flow



```
                    +---------+
```

3.3 Run states

State	Meaning	Terminal?
`RECEIVED`	CLI or GUI request received.	no
`SPEC_VALIDATED`	Specification parsed and constraints are structurally valid.	no
`GENERATING`	Scenarios and candidate records are being attempted.	no
`DEGRADED`	Optional LLM is unavailable and a declared fallback is active, or generation is continuing with bounded rejections.	no
`COMPLETED`	Requested accepted record count written and manifest/report finalized.	yes
`EXHAUSTED`	Attempt budget reached before requested count.	yes
`REJECTED`	Specification or CLI input invalid.	yes
`FAILED`	Unrecoverable writer, calculator, or dependency failure.	yes

A run MUST terminate after `max_attempts`, even when validation or deduplication rejects every candidate. A rejected candidate MUST NOT consume an accepted-record sequence number.

3.4 Deterministic versus probabilistic boundary

The specification parser, seeded sampling, constraint evaluation, truth engine, schema validation, normalization, exact deduplication, reports, and manifests are deterministic. Only the optional language realization adapter is probabilistic. The LLM MUST receive an immutable scenario and MUST NOT receive authority over truth fields.

---

4. Interfaces / contracts

C-01 Dataset specification

```yaml
dataset:
  name: mortgage_questions
  domain: mortgage
  size: 1000
  seed: 42
  max_attempts: 5000
  output: mortgage_questions.jsonl
  report: mortgage_questions.report.json
  manifest: mortgage_questions.manifest.json

schema:
  fields:
    - question
    - intent
    - principal
```

```
    - annual_rate
    - term_years
    - payment
    - expected_outcome

categories:
  payment:
    weight: 0.20

constraints:
  - principal > 0

realization:
  method: template
  max_regenerations: 3
```

Required top-level sections are `dataset`, `schema`, and `categories`. `dataset.domain` MUST name a registered C-10 calculator. `constraints` and `realization` MAY be omitted only when the defaults in this specification are used: no additional constraints and `method: template`, `max_regenerations: 3`. The `schema.fields` section MUST use typed descriptors as defined in C-01a.

C-01a Typed field descriptors

```
schema:
  fields:
    - name: question
      type: string
      required: true
      nullable: false
    - name: intent
      type: enum
      values: [payment, principal, term, rate]
      required: true
      nullable: false
    - name: principal
      type: decimal
      required: false
      nullable: true
      minimum: 0
    - name: annual_rate
      type: decimal
      required: false
      nullable: true
      minimum: 0
    - name: term_years
      type: decimal
      required: false
      nullable: true
      minimum: 0
    - name: payment
      type: decimal
      required: false
      nullable: true
      minimum: 0
    - name: expected_outcome
```

```
    type: enum
    values: [calculated, clarification, unsupported_scope, payment_too_low]
    required: true
    nullable: false
```

Every field descriptor MUST define `name`, `type`, `required`, and `nullable`; numeric fields MAY define `minimum`, `maximum`, and `distribution`; enum fields MUST define finite `values`. The generated record schema MUST use the Chapter 31 shape in C-05: `case_id`, `category`, `question`, `expected`, and optional `metadata`. Unknown fields are rejected.

C-02 Scenario

```python
@dataclass(frozen=True)
class Scenario:
    scenario_id: str
    category: str
    fields: dict[str, object]
    expected_outcome: str
    seed_offset: int
```

`fields` MUST contain only schema-declared fields. `scenario_id` MUST be stable within a seeded run and MUST NOT depend on wall-clock time.

C-03 Ground truth

```python
@dataclass(frozen=True)
class GroundTruth:
    outcome: str
    fields: dict[str, object]
    source: str
    calculator_version: str
```

The ground-truth engine MUST be deterministic for the same scenario and calculator version. It MUST return a structured error for invalid or unsupported scenarios.

C-04 Realization

```python
@dataclass(frozen=True)
class Realization:
    question: str
    method: str                 # template | ollama
    template_id: str | None
    model: str | None
    raw_response: str | None
```

A realization MUST contain a non-empty question. `raw_response` MUST be omitted from normal persisted datasets and MAY be retained in a bounded debug artifact.

C-05 Dataset record

The published JSONL record MUST match the Chapter 31 evaluation-case shape:

```
{
  "case_id": "principal-affordability-06",
  "category": "principal",
  "question": "I can afford $3,000 per month at 6% for 30 years. How much can I borrow?",
  "expected": {
    "intent": "principal",
```

```
    "outcome": "calculated",
    "fields": {"payments": 360, "payment": "3000"},
    "result": {"principal": "500000", "tolerance": "1000"}
  },
  "metadata": {
    "scenario_id": "principal-000006",
    "generator": "template",
    "template_id": "principal_07",
    "seed": 183729,
    "spec_hash": "sha256:..."
  }
}
```

Required fields are `case_id`, `category`, `question`, and `expected.outcome`. `expected.intent`, `expected.fields`, and `expected.result` are required when declared by the specification. `metadata` is required for generated datasets and MUST identify scenario and realization provenance. Ground truth is represented by `expected`, never by the language model.

C-06 Distribution and constraint contracts

```yaml
principal:
  distribution: uniform
  min: 100000
  max: 2000000
annual_rate:
  distribution: uniform
  min: 0.03
  max: 0.09
term_years:
  distribution: values
  values: [10, 15, 20, 25, 30]
```

```python
class Distribution(Protocol):
    def sample(self, rng: Random) -> object: ...

class Constraint(Protocol):
    def check(self, scenario: Scenario) -> tuple[bool, str]: ...
```

Supported v0.1 distributions and semantics are pinned:

- `uniform`: sample from `[min,max)` using `rng.random()`; `min <= max`.
- `lognormal`: sample `exp(rng.normalvariate(mu, sigma))`; `sigma > 0`; result MUST be checked against optional bounds.
- `choice`: select one value using declared finite `values` and optional normalized `weights`.
- `values`: select one item from a finite ordered list using the seeded RNG.

All numeric samples MUST be converted to the declared field type after sampling. Category selection MUST use one ordered cumulative-weight draw per candidate; categories are traversed in specification order; weights MUST sum to `1.0` within `1e-9`. A distribution MUST have finite bounds or finite values.

Constraint expressions MUST use the safe grammar `expression := comparison (("and" | "or") comparison)*`; comparisons are `identifier | literal` joined by `> >= < <= == != in`; arithmetic permits only `+ - * /` and parentheses. Identifiers resolve only to scenario fields. Function calls, attribute access, indexing, imports, assignment, comprehensions, and unrestricted `eval` are forbidden. Missing fields and division by zero are deterministic constraint failures.

C-07 Validator result

```python
@dataclass(frozen=True)
class ExtractionResult:
    valid: bool
    fields: dict[str, object]
    reasons: tuple[str, ...]

@dataclass(frozen=True)
class ValidationResult:
    valid: bool
    stage: str                  # schema | semantic | scope | dedup
    reasons: tuple[str, ...]
    extracted: dict[str, object]
```

Validation MUST report all known reasons in deterministic order. A valid record requires schema validity, scenario equivalence, declared outcome preservation, and scope compliance. Regeneration MUST derive `candidate_seed` as a stable hash of `(run_seed, scenario_id, attempt_index, realization_method)`; the same candidate attempt MUST therefore be reproducible.

C-08 Quality report

```json
{
  "report_version": "0.1",
  "dataset": "mortgage_questions",
  "requested": 1000,
  "attempted": 1016,
  "accepted": 984,
  "rejected": 32,
  "complete": false,
  "category_counts": {},
  "validation": {"valid": 984, "rejected": 32},
  "duplicates": {"exact": 3, "near": 7},
  "realization_methods": {"template": 1016},
  "metrics": {"acceptance_rate": 0.9685},
  "failures": [],
  "manifest": "manifest.json"
}
```

Metric definitions are fixed: `acceptance_rate = accepted / attempted`; category percentage is `accepted_in_category / accepted`; validation rejection rate is `rejected / attempted`; duplicate rates use attempted candidates as denominator; unavailable cost is `null`, not zero. Percentages and rates MUST serialize with six decimal places. Empty denominators serialize as `null`.

C-09 Reproducibility manifest

```json
{
  "manifest_version": "0.1",
  "generator_version": "0.1.0",
  "spec_hash": "sha256:...",
  "dataset_path": "mortgage_questions.jsonl",
  "dataset_sha256": "sha256:...",
  "report_path": "mortgage_questions.report.json",
  "manifest_path": "mortgage_questions.manifest.json",
  "seed": 42,
  "requested_size": 1000,
```

```
  "max_attempts": 5000,
  "adapter": "template",
  "model": null,
  "temperature": null,
  "created_at": "ISO-8601 timestamp"
}
```

`reproduce MANIFEST` MUST resolve `dataset_path`, `report_path`, and `manifest_path` relative to the manifest directory unless an explicit `--dataset` override is supplied. It MUST compare `dataset_sha256` after normalizing only explicitly allowed nondeterministic metadata. A hash mismatch is a deterministic failure. The manifest timestamp is provenance only and is excluded from byte-identity comparisons. Readers MUST accept exactly `manifest_version="0.1"` and `report_version="0.1"`; a version mismatch MUST fail unless the command provides an explicit `--force`, which emits a warning.

C-10 Domain calculator registry and protocol

```python
class GroundTruthCalculator(Protocol):
    @property
    def version(self) -> str: ...

    def calculate(self, scenario: Scenario) -> GroundTruth: ...

@dataclass(frozen=True)
class GroundTruthError:
    code: str
    message: str
    field: str | None
    details: dict[str, str]

class CalculatorRegistry(Protocol):
    def get(self, name: str) -> GroundTruthCalculator: ...
```

`dataset.domain` MUST select a registered calculator. Unknown domains are specification errors. Calculator failures MUST use `GroundTruthError` with stable `code`, `message`, `field`, and `details`. The generator MUST use dependency injection; a domain calculator MUST NOT receive an LLM client.

---

5. Interface specification

5.1 CLI (`synthgen`), primary surface

Subcommand	Behavior	Exit
`synthgen generate SPEC`	Validate spec, generate accepted records, write JSONL/report/manifest, and print a summary.	`0` completed; `1` attempt budget exhausted; `2` usage/spec error; `5` model/dependency failure.
`synthgen validate DATASET`	Validate JSONL schema, semantic fields, scope, and metadata without generating new records.	`0` valid; `1` invalid records; `2` usage/input error.
`synthgen stats DATASET`	Print category, outcome, validation, duplicate, and realization statistics.	`0` success; `2` usage/input error.

Subcommand	Behavior	Exit
`synthgen preview SPEC`	Generate a bounded number of records without publishing a dataset.	`0` success; `2` usage/spec error; `5` model failure.
`synthgen reproduce MANIFEST`	Re-run the recorded configuration and compare normalized output to the referenced dataset.	`0` identical; `1` mismatch; `2` invalid manifest/input.
`synthgen inspect DATASET`	Print sample records, metadata, rejected-record summary, and report references without changing files.	`0` success; `2` usage/input error.

Required options:

```
--size N              override requested accepted-record count
--seed N              override the specification seed
--output PATH         JSONL output path
--report PATH         quality report path
--manifest PATH       reproducibility manifest path
--method template|ollama
--model MODEL         Ollama model for --method ollama
--host URL            Ollama endpoint
--max-attempts N      bounded candidate-attempt budget
--allow-partial       publish a report/manifest marked complete=false after exhaustion
--force               admit compatible-version warnings during inspect/reproduce
--verbose [INFO|DEBUG]
```

`--verbose` MAY appear before or after the subcommand. Bare `--verbose` equals `INFO`. Omitted verbosity is quiet. `INFO` emits metadata only to stderr; `DEBUG` additionally emits raw model prompts/responses to stderr. Normal dataset/report output is not mixed with diagnostics. `generate` MUST default `--output`, `--report`, and `--manifest` from the `dataset.output`, `dataset.report`, and `dataset.manifest` fields; explicit CLI paths override those defaults.

5.2 GUI (`synthgen-gui`), optional bounded surface

The GUI is out of v0.1 acceptance. If implemented, it MUST provide specification selection, size/seed/method controls, preview, generate, validate, stats, and inspect actions. It MUST use the contracts in §4 and MUST not duplicate scenario, truth, constraint, or validation logic. It MUST provide `Off`, `INFO`, and `DEBUG` diagnostics, default `Off`; raw model prompts/responses MAY appear only at DEBUG in a dedicated diagnostics view. When realization method `ollama` is selected, the GUI MUST reveal model and host controls defaulting to `llama3.2` and `http://127.0.0.1:11434`; Preview MUST pass those values to the shared service and then to `OllamaRealizer`. Template mode MUST pass no Ollama override.

5.3 Output rules

`generate` MUST write accepted JSONL records in accepted sequence order. Report and manifest writes MUST use UTF-8, sorted JSON keys, and a trailing newline. Writes MUST first land in a run-specific staging directory and MUST be atomically renamed into the requested paths only after all required artifacts validate. Without `--allow-partial`, any failure publishes none of the three artifacts. With `--allow-partial`, the report and manifest MUST set `complete=false`, the JSONL contains accepted records only, and `reproduce` MUST reject it as a complete match. If any required artifact cannot be written, the run MUST return a failure and MUST NOT print a completed-success summary.

---

6. Invariants (must hold in every valid implementation)

ID	Invariant
I-001	The same specification hash, generator version, seed, adapter/model configuration, and generation options produce the same normalized dataset on the deterministic path.
I-002	Ground truth is calculated only from the structured scenario through C-10; language generation cannot modify it.
I-003	Every accepted record has exactly one scenario, one ground truth, one realization, and one validation result.
I-004	Every accepted record satisfies schema, semantic equivalence, declared scope, and deduplication gates.
I-005	Category selection follows declared weights within the configured sampling policy; the realized counts and deviations are reported rather than hidden.
I-006	Every random draw comes from the run-local seeded RNG; global random state MUST NOT affect output.
I-007	A rejected candidate cannot appear in the published JSONL and cannot consume an accepted record ID.
I-008	Every report failure identifies a deterministic stage and reason; no rejection is silently discarded.
I-009	Exact duplicate detection is based on the specified normalized question representation and is deterministic.
I-010	Optional LLM output is untrusted: it cannot change ground truth, category, expected outcome, or acceptance status without validator evidence.
I-011	Omitted verbosity is quiet; INFO excludes raw model content; DEBUG is the only raw-payload level; diagnostics cannot alter artifacts.
I-012	A manifest is sufficient to identify the generator configuration and compare a reproduced run, with timestamp normalization applied.
I-013	Every accepted record remains within the declared domain scope; unsupported concepts are rejected.

---

7. Constraints (precise and measurable)

ID	Constraint
K-01	Python MUST be `>=3.12,<3.13`; installation and commands MUST use `uv`.
K-02	`max_attempts` MUST be finite and MUST default to `max(size * 5, size + 10)`.
K-03	`preview` MUST accept at most 100 records and MUST NOT publish a production dataset by default.
K-04	Numeric distributions MUST declare finite bounds or finite values.
K-05	LLM realization retries MUST be bounded by `max_regenerations`, default `3`; repeated failure rejects the candidate.
K-06	Raw model prompt/response excerpts MUST be omitted by default and, when explicitly retained, capped at 4,000 characters per candidate.
K-07	All generated dataset, report, and manifest writes MUST be UTF-8 JSON/JSONL with stable key ordering and a trailing newline.
K-08	Exact deduplication MUST run before publication; near deduplication MAY be disabled only when the report records that it was disabled.
K-09	The deterministic/template path MUST open no network sockets.
K-10	`reproduce` MUST compare normalized records while excluding only the manifest timestamp and other explicitly listed nondeterministic fields.

ID	Constraint
K-11	A specification MUST contain at least one category and category weights MUST be positive and sum to `1.0` within `1e-9`, unless count allocation is used explicitly.
K-12	A single record MUST NOT contain more than 1 MB of language or metadata payload.

---

8. Edge cases and failure semantics

ID	Case	Semantics
E-01	Missing or unreadable specification	Exit `2`; write no dataset artifacts.
E-02	Malformed YAML/JSON	Exit `2` with file/line diagnostic.
E-03	Unknown distribution, category, constraint, or realization method	Exit `2`; do not silently use a default.
E-04	Impossible constraint set	Exit `1` after bounded attempts or `2` during static satisfiability validation, with the chosen stage recorded.
E-05	Ground-truth calculator rejects a scenario	Reject candidate, record calculator stage/reason, continue until attempt budget.
E-06	Template produces a schema-invalid or semantically mismatched record	Reject or regenerate within `max_regenerations`; record every reason.
E-07	LLM produces malformed output, changes semantics, or introduces unsupported scope	Reject candidate; never alter ground truth; retry only within bound.
E-08	Ollama unavailable	Real run records model failure and exits `5`; template/mock run remains offline and unaffected.
E-09	Duplicate candidate	Reject as exact/near duplicate, record duplicate key and prior record ID, continue.
E-10	Requested size cannot be reached before `max_attempts`	Write a partial report only if explicitly allowed by `--allow-partial`; otherwise exit `1` without claiming a complete dataset.
E-11	Empty dataset	`validate`, `stats`, and `inspect` return deterministic input errors; `generate --size 0` is a usage error.
E-12	Seed omitted	Use the specification seed if present; otherwise require an explicit CLI seed for reproducibility.
E-13	Conflicting CLI overrides	Exit `2`; CLI MUST NOT silently choose between conflicting size, seed, or method declarations.
E-14	Near-deduplication unavailable	Continue only if the report records `near_deduplication: disabled`; exact deduplication remains mandatory.
E-15	Artifact write failure	Exit `5`; report the path and preserve no misleading success message.
E-16	Raw diagnostic request	Only `DEBUG` or explicit `--include-raw` may expose raw model content; default and INFO artifacts MUST omit it.
E-17	Artifact version mismatch	`inspect`, `stats`, `validate`, and `reproduce` MUST reject a version other than `0.1` unless `--force` is supplied; forced reads emit a warning.
E-18	Unknown domain/calculator	Reject before scenario generation with a structured specification error; no generic truth fallback is permitted.

ID	Case	Semantics
E-19	Constraint expression violation or unsafe syntax	Reject the candidate with a deterministic constraint-stage reason; never execute arbitrary code.

---

9. Acceptance criteria, tests, and evals

9.1 Specification and distributions

- **T-01** Valid YAML loads into C-01 and produces a deterministic normalized specification.
- **T-02** Malformed YAML/JSON, unknown methods, missing required sections, invalid weights, and unbounded distributions return exit 2 with actionable diagnostics.
- **T-03** Same seed/spec/options produce identical normalized scenario sequences; different seeds produce a detectable sequence difference.
- **T-04** Weighted category allocation reports actual counts and deviations from requested weights.

9.2 Scenario and ground truth

- **T-05** Valid mortgage scenarios satisfy declared domain constraints and produce deterministic ground truth.
- **T-06** Deliberately invalid payment scenarios produce `payment_too_low` ground truth without an LLM call.
- **T-07** A calculator double can be substituted through C-10 without changing generator or validator code.
- **T-08** Ground truth remains byte-identical when the language realization method changes from template to Ollama.

9.3 Language realization and validation

- **T-09** Template realizations preserve intent, fields, units, and expected outcome across every template variation.
- **T-10** An LLM realization that changes a principal, rate, term, payment, or outcome is rejected with semantic reasons.
- **T-11** Malformed LLM output is boundedly retried and then rejected; no malformed record is published.
- **T-12** Unsupported concepts such as taxes, insurance, HOA fees, adjustable-rate mortgages, or lender advice are rejected when outside the specification.
- **T-13** Exact and near-duplicate decisions are reported with the normalized key and prior record reference.

9.4 Dataset schema and artifacts

- **T-14** Every accepted C-05 record has separate input, ground-truth, and metadata objects.
- **T-15** JSONL output has one valid JSON object per line, stable key ordering, UTF-8 encoding, and a trailing newline.
- **T-16** Quality reports contain accepted/rejected counts, category counts, validation counts, duplicate counts, realization methods, and failure records.
- **T-17** Manifests contain generator version, spec hash, seed, size, attempt budget, adapter/model, and timestamp.
- **T-18** `reproduce` resolves manifest-relative paths, verifies `dataset_sha256`, normalizes timestamps, and confirms identical deterministic records; a changed seed, spec, or dataset fails comparison.
- **T-18a** Manifest version mismatch is rejected without `--force` and admitted only with an explicit warning using `--force`.

9.5 CLI

- **T-19** `generate` creates JSONL, report, and manifest and returns `0` when the requested size is reached.
- **T-20** `validate` returns `1` for invalid records and identifies record IDs and validation stages.
- **T-21** `stats` and `inspect` are read-only and do not modify the dataset.
- **T-22** `preview` is bounded at 100 records and does not publish production artifacts unless explicitly requested.
- **T-23** `reproduce` returns `0` for an identical normalized run and `1` for a mismatch.
- **T-24** Dataset exhaustion returns `1` and records the final rejection reason rather than looping forever.
- **T-24a** Atomic staging publishes no artifacts on an ordinary failure and publishes `complete=false` only with `--allow-partial`.

9.6 Verbosity and observability

- **T-25** Omitted verbosity produces no diagnostic logs in normal output.
- **T-26** Bare `--verbose` and `--verbose INFO` emit metadata only to stderr; generated stdout/artifacts are unchanged.
- **T-27** `--verbose DEBUG` emits clearly labeled raw prompts/responses for Ollama realization and never emits them at INFO.
- **T-28** GUI `Off`/`INFO`/`DEBUG` diagnostics match the CLI semantics and do not modify generated records.
- **T-28a** Constraint evaluation accepts only the pinned safe grammar and rejects calls, attribute access, imports, and unrestricted expressions.

9.7 GUI (optional acceptance surface)

- **T-29** If `synthgen-gui` is implemented, it loads a specification and displays categories, distributions, and constraints without performing generation on construction.
- **T-30** GUI Generate and Preview invoke the same service as the CLI and display report/artifact paths.
- **T-30a** Malformed field descriptors, unregistered calculators, malformed calculator errors, and unsupported domains fail deterministically before generation.

9.8 Manual evaluation

- **T-31** Generate a 30-record mortgage-question dataset using templates and inspect category/outcome distributions.
- **T-32** Generate the same dataset twice with the same seed and compare normalized JSONL bytes.
- **T-33** Run one Ollama realization batch and confirm ground truth is unchanged when the model produces paraphrases or malformed output.
- **T-34** A record generated from a Chapter 31-style sample preserves `case_id`, `category`, `question`, nested `expected`, and optional provenance metadata exactly as specified.
- **T-35** `--include-raw` is the only report option that persists a bounded raw model-response excerpt; default reports contain no raw model content.

---

10. Dependencies and environment

Concern	Decision	Rationale
Python	`>=3.12,<3.13`	Reproducible runtime.
Environment	`uv`	Dependency and command reproducibility.
Config formats	YAML and JSON	Human-authored declarative specifications.
CLI	Standard library `argparse` or equivalent	Offline, scriptable primary interface.

Concern	Decision	Rationale
Deterministic RNG	Python `random.Random(seed)`	Run-local reproducibility without global-state coupling.
Ground truth	Injected domain calculator protocol	Keeps the generator domain-independent.
Templates	In-repository template functions/data	Correctness-first language generation.
Optional model	Local Ollama adapter	Linguistic diversity without hosted-service dependency.
Testing	`pytest`; optional `hypothesis`	Unit, boundary, and generated-case tests.
GUI	Optional PyQt5 + `pytest-qt`	Shared with the bootcamp's existing desktop labs.
Logging	Loguru or equivalent configured logger	Colored CLI diagnostics and levelled GUI diagnostics.

Expected project structure:

```
labs/week4/chapter32/
+-- SPEC.md
+-- README.md
+-- pyproject.toml
+-- schemas/
|   +-- dataset-spec.json
|   +-- dataset-record.json
|   +-- report.json
|   +-- manifest.json
+-- examples/
|   +-- mortgage.yaml
+-- evals/
|   +-- generated.jsonl
|   +-- report.json
|   +-- manifest.json
+-- src/synthgen/
|   +-- __init__.py
|   +-- cli.py
|   +-- spec.py
|   +-- schema.py
|   +-- scenarios.py
|   +-- distributions.py
|   +-- constraints.py
|   +-- truth.py
|   +-- templates.py
|   +-- llm.py
|   +-- validators.py
|   +-- dedup.py
|   +-- metrics.py
|   +-- writers.py
|   +-- diagnostics.py
|   +-- ui.py
+-- tests/
    +-- test_spec.py
    +-- test_scenarios.py
```

```
    +-- test_constraints.py
    +-- test_truth.py
    +-- test_generators.py
    +-- test_validation.py
    +-- test_dedup.py
    +-- test_reports.py
    +-- test_cli.py
    +-- test_ui.py
```

Reproducibility commands:

```
uv sync --extra test
uv run pytest
uv run synthgen generate examples/mortgage.yaml --size 1000 --seed 42 --output evals/generated.jsonl
uv run synthgen validate evals/generated.jsonl
uv run synthgen stats evals/generated.jsonl
uv run synthgen inspect evals/generated.jsonl
uv run synthgen reproduce evals/manifest.json
```

---

11. Traceability matrix (id -> where realized)

```
R-01/R-05  --> spec.py + distributions.py + constraints.py --> T-01..T-04
R-02/R-03  --> scenarios.py + truth.py + C-10 --> T-05..T-08
R-04/R-23  --> scenarios.py + validators.py --> T-06, T-12
R-06/R-07  --> templates.py + llm.py --> T-09..T-11, T-33
R-08/R-09  --> validators.py + regeneration policy --> T-09..T-12, T-24
R-10       --> dedup.py --> T-13
R-11/R-12  --> writers.py + C-05/C-09 --> T-14..T-18
R-13       --> metrics.py + report writer --> T-16
R-14/R-15  --> cli.py --> T-19..T-24
R-16/R-17  --> seeded RNG + llm.py --> T-03, T-08, T-33
R-18       --> schema.py + validators.py + writers.py --> T-02, T-11, T-24
R-20       --> ui.py + pyproject.toml --> T-29, T-30
R-21       --> diagnostics.py + cli.py + ui.py --> T-25..T-28
R-22       --> truth.py calculator protocol --> T-07
R-24       --> C-05/C-08/C-09 metadata --> T-14..T-18
F-001      --> C-09 manifest paths/hashes + reproduce --> T-18, T-18a
F-002      --> C-06 distributions/category allocation --> T-02..T-04
F-003      --> C-01a typed record/schema descriptors --> T-01, T-02, T-34
F-004      --> C-06 safe constraint grammar --> T-28a
F-005      --> C-10 calculator registry/error --> T-30a
F-006      --> C-07 extraction + candidate seed derivation --> T-09..T-11, T-03
F-007      --> §5.3 staging/partial artifact protocol --> T-24a
F-008      --> C-08 metric definitions --> T-04, T-16
F-009      --> §5.2 explicit v0.1 GUI boundary --> T-29, T-30
F-010      --> §5.3 version compatibility --> T-18a
R-25       --> C-09/C-08 provenance and artifact identity --> T-17, T-18, T-24a
I-001/I-006 --> seeded run RNG + reproduce --> T-03, T-18, T-32
I-002/I-010 --> truth.py + llm.py boundary --> T-07, T-08, T-10
I-003/I-004/I-007 --> validators.py + writers.py --> T-09, T-14, T-15
I-005       --> distributions.py + metrics.py --> T-04, T-16
I-008/I-009 --> report.py + dedup.py --> T-13, T-16
I-011       --> diagnostics.py --> T-25..T-28
I-012       --> manifest.py/reproduce --> T-17, T-18
```

```
I-013       --> scope validator --> T-12
K-01..K-12  --> pyproject.toml + bounded services --> T-01..T-30
E-01..E-16  --> cli.py + validators.py + writers.py --> T-02, T-11, T-20, T-24
C-01..C-10  --> src/synthgen modules --> T-01..T-18, T-30a, T-34, T-35
```

End of specification. This document is the source of truth; implementation and tests MUST be derived from it and kept synchronized with the traceability matrix.